

CUDA

27/09/26

⇒ % Write file hello.cu;

This will store the below things into a file named hello.cu.

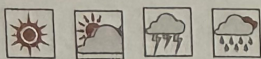
⇒ nvcc hello.cu → hello.

↓
Cuda
compiler

↓
Program
we stored
as
file

↓
hello.cu be compiled
into hello
which is an
executable
program.

To increase effectiveness, make your emotions subordinate to your commitments



Mo Tu We Th Fr Sa Su

Date / /

Now, to run the code,

!./hello

Why not -> !./hello.cu?

Because this is the source code, whereas after compilation, it turns to executable code.

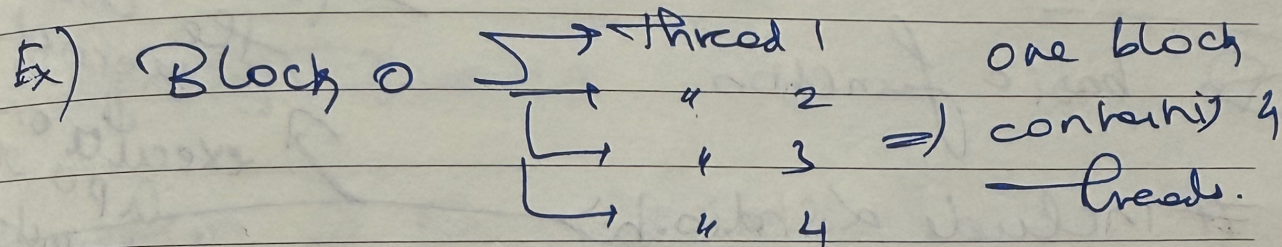
global -:

It tells the compiler that this function is a GPU kernel, i.e., a function to be executed by GPU threads. We write this before any function which needs to be run using

threadIdx.x

thread is one execution instance of our kernel.

Block is a group of threads.



We organize these threads into blocks for better structure for GPU.

So, when we write `helloFromGPU <<< 1, 4 >>>()`

running the function

`<<< 1, 4 >>>`

(Number of Blocks) (threads per Block) $\Rightarrow$ 1 Block, 4 threads

Threads b/w one block has shared Memory.

~~Therefore, threads x prints the function from each~~

Hence, we launch a function on 1 Block of threads. 1 thread instances are created

The first step to getting the things you want out of life is this: Decide what you want.

for the same function.

So basic function

#include <stdio.h>

global void HelloFromGpu() {

printf ("Hello from Gpu! Thread id: %d\n",
threadIdx.x);

}

int main () {

HelloFromGpu (<<< 1, 4 >>>) ;

cuda Device Synchronize();

return 0;

}

It tells the cpu, wait
till Gpu finishes all
the work it has.

this is called
the CUDA
kernel.
↓
executed on
GPU when
launched
from
CPU.